Engenharia de IA

Anotações de estudo da trilha da equipe

Gabriel

29/10/2026

Sumário

Prefácio **3**
Como ler 3
Estrutura 3

1 Segurança **4**
1.1 **Modelo de ameaça em sistemas com agentes** 4

Prefácio

Este livro é o registro do meu estudo da trilha de Engenharia de IA da minha equipe. Ele acompanha o roadmap da trilha: cada capítulo corresponde a um tema do roadmap e junta o que li, o que entendi e as dúvidas que ficaram.

Como ler

As anotações são de estudo, não de referência oficial. Cada tópico traz o link da fonte original (artigo, documentação, vídeo), e vale ir até ela quando o resumo não bastar. Onde ficou uma dúvida em aberto, ela está marcada no texto como *dúvida*, e é provável que eu a resolva mais adiante.

Estrutura

O livro cresce conforme avanço no roadmap, e os capítulos seguem a ordem em que estudo cada tema. Os capítulos escritos até agora são:

1. **Segurança**

Os demais temas da trilha entram como novos capítulos.

1 Segurança

1.1 Modelo de ameaça em sistemas com agentes

Prompt injection explained

- https://simonw.substack.com/p/prompt-injection-explained-with-video
- "The hardest problem in computer science is convincing AI enthusiasts that they can't solve prompt injection vulnerabilities using more AI." ~ Simon Wilson
  - *prompt begging*
- Dois approaches
  - IA para detectar se existe má intenção no Input
  - IA para detectar se existe má intenção no Output
    - Poderia utilizar um classificador aqui *(JEV)*
  - Contudo, *"Segurança baseada em probabilidade, não é segurança no fim das contas"*
- ***dual language model pattern***
  - Um agente com privilégios;
  - Um apenas com leitura

The lethal trifecta for AI agents

- https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/
- Acess to **Private Date**
- Ability to **Externally Communicate**
- Exposure to **Untrusted Content**

LLM01:2025 Prompt Injection

- https://genai.owasp.org/llmrisk/llm01-prompt-injection/
- Dois tipos:
  - Injeção de Prompt Direta
  - Injeção de Prompt Indireta
- Por ser estocástico, não é possível afirmar que existe uma forma infalível de mitigação
- Estratégias:
  1. Restringir o comportamento do modelo
  2. Definir e validar formato de saída esperado
  3. Filtragem na **entrada** e **saída**
     1. Aplicar filtros semânticos
     2. Verificação de strings para detectar conteúdo não permitido
     3. Avaliar respostas usando tríade RAG
  4. Aplicar princípio do Menor Privilégio
  5. Aprovação humana para casos extremos
  6. Segregar e identificar conteúdo externo
  7. Realizar testes regularmente

LLM06:2025 Excessive Agency

- https://genai.owasp.org/llmrisk/llm062025-excessive-agency/
- Quando Agente tem mais acesso do que precisa para realizar ações
  - Funcionalidades excessivas
  - Permissões excessivas
  - Autonomia excessivas

1. **Funcionalidades Excessivas**
   - Um agente que deveria ter a habilidade de ler documento, mas tem acesso a uma extensão que também permite a alteração e “delete” dos documentos
2. **Permissões Excessivas**
   - Um agente que deveria **apenas** ler informações de um banco, possui permissão para dar UPDATE, INSERT and DELETE
   - Um agente com permissão de ver todos os arquivos dos usuários de um repositório

3. **Autonomia Excessiva**
    - O agente ser capaz de tomar decisões de alto impacto sem necessidade de aprovação de usuário
        - Exclusão de documentos

ESTRATÉGIAS MITIGAÇÃO

- Minimizar extensão
    - Se o agente não precisa da capacidade de realizar uma função, ela não deve ser fornecida ao agente
- Minimizar funcionalidade extensão
- Evitar *"open ended extensions"*
    - Se o agente deve escrever um arquivo, o ideal seria evitar algo que de abertura *ao shell*
        * Com acesso, pode fazer várias outras coisas
        * Dar acesso a algo que vai apenas ser capaz de escrever
- Minimizar permissão de extensão
- Executar extensão no contexto de usuário
- Requirir aprovação humana
    - *Human in the loop*
- Mediação completa (*dúvida*)
- Higienizar entrada e saída LLM

How Microsoft defends against indirect prompt injection attacks

- https://www.microsoft.com/en-us/msrc/blog/2025/07/how-microsoft-defends-against-indirect-prompt-injection-attacks
- (*blog fala mais sobre injeção de prompt indireta*)

COMO SE DEFENDER CONTRA INJEÇÃO INDIRETA - *Microsoft*

1. ***Prevenir a injeção***
    1. *System Prompt Endurecido*
        - No *System Prompt*, dizer explicitamente para não tratar conteúdo externo como ordens
        - Ajuda a reduzir taxa de ataques

        - Contudo, probabilístico

    2. *Spotlighting*

        - *"Analogia estagiário recebendo cartas cliente e chefe misturadas"*

            - Pintar cartas cliente

        - Transforma texto externo de forma reconhecível
        - Avisa no system prompt que aquilo **nunca** é instrução
        - Formas de separar:

            - Delimiting
            - Datamarking
            - Encoding

2. ***Detecção da injeção***

    - Roda antes que injeção possa influenciar resposta

    1. *Prompt Shields*

        - "*Detector de metal"*
        - Classificador ML (*até o momento*) treinado para detectar coisa do tipo
        - Probabilístico, pode falhar

    2. *Integração Defender for Cloud*

3. ***Prevenir o impacto***

    - Não desenham para prevenir todas as injeções

        - Quando uma injeção passar, ela não deve funcionar

    - Exemplo de mitigações como stack canaries, ASLR, CFG e DEP

    1. *Governança de Dados e Permissões*

        - Determinístico
        - "*Estagiário pode ser enganado com cartas falsas, mas o crachá dele não abre todas as portas*"
        - Limitar permissões

    2. *Bloqueio determinístico das técnicas de exfiltração*

    3. *Human in the Loop*